

PROGRAMMATION FONCTIONNELLE 2 : TD6

février 2026 — Pierre Rousselin

On garde dans ce TD les opérations et notations du TD précédent.

Exercice 1 : injection

Dans chacun des cas suivants, donner l'état de la preuve après la tactique **injection** et terminer la preuve.

1. **Lemma** `ex1_1` (`n m : nat`) : `S (S (S n)) = S (S m) -> S n = m`.
Proof. `intros H. injection H.`
2. **Lemma** `ex1_2` (`n m : nat`) : `2 + (S n) = S (S m) -> S n = m`.
Proof.
`intros H. injection H.`
3. **Lemma** `ex1_3` (`l1 l2 : list nat`) (`n : nat`) : `1 :: 2 :: l1 = 1 :: n :: l2 -> 2 = n /\ l1 = l2`.
Proof.
`intros H.`
`injection H as Hn Hl.`
4. **Lemma** `ex1_4` (`l1 l2 : list nat`) :
`[1; 2; 3] ++ l1 = 1 :: 2 :: l2 -> 3 :: l1 = l2`.
Proof.
`intros H. injection H.`

--- * ---

Correction de l'exercice 1 :

1. `H` devient `S n = m`. `exact H`.
2. idem, après réduction de `2 + S n` en `S (S (S n))`.
3. L'égalité inutile `1 = 1` n'apparaît pas, on a `Hn : 2 = n` et `Hl : l1 = l2`. `split; assumption` termine la preuve.
4. Après réduction de `app`, `H` a la forme donnée en conclusion. `exact H` termine la preuve.

--- * ---

Exercice 2 : discriminate

Dans chaque cas suivant, réduire les termes dans l'égalité fournie à **discriminate** ainsi que le ferait Rocq et dire si la tactique **discriminate** donnée réussit ou échoue.

1. **Lemma** `ex2_1` : `4 <> 2`.
Proof. `intros H. discriminate H.`
2. **Lemma** `ex2_2` (`n : nat`) : `2 + n <> 1`.
Proof.
`intros H. discriminate H.`
3. **Lemma** `ex2_3` (`n : nat`) : `S (S n) <> 2`.
Proof.
`intros H.`
`discriminate H.`
4. **Lemma** `ex2_4` (`n : nat`) : `n + 2 <> 1`.
`intros H.`
`discriminate H.`
5. **Lemma** `ex2_5` (`l : list nat`) : `42 :: l <> []`.
Proof.
`intros H.`
`discriminate H.`

```
6. Lemma ex2_6 : negb (negb true) <> false.
Proof.
  intros H.
  discriminate H.
```

--- * ---

Correction de l'exercice 2 :

1. Pour Rocq, c'est $S (S (S 1)) = S (S 0)$, après simplification, ça donne $S 1 = 0$, `discriminate` termine la preuve.
2. $2 + n$ est réduit en $S (S n)$ donc l'égalité, après simplification, devient $S 1 = 0$, `discriminate` termine la preuve.
3. erreur, H peut être simplifié en $n = 0$, ce qui n'est pas contradictoire
4. erreur, Rocq ne peut pas réduire $n + 2$ en quoi que ce soit donc `discriminate` ne peut pas avancer
5. termine la preuve, `cons` n'est jamais égal à `nil`
6. après réduction, on obtient `true = false`, donc égalité entre constructeurs différents, termine la preuve

--- * ---

Exercice 3 : Discriminate pour éliminer les cas impossibles

Écrire la preuve de chacun des lemmes suivants avec une preuve par cas, en utilisant `discriminate` pour éliminer les cas impossibles.

1. Lemma `negb_eq_true` ($b : \text{bool}$) : `negb b = true` $\rightarrow$ `b = false`.
2. Lemma `add_eq_0` ($n\ m : \text{nat}$) : `n + m = 0` $\rightarrow$ `n = 0` $\wedge$ `m = 0`.
3. Lemma `app_eq_nil` $\{A : \text{Type}\}$ ($l1\ l2 : \text{list } A$) :
`l1 ++ l2 = []` $\rightarrow$ `l1 = []` $\wedge$ `l2 = []`.

--- * ---

Correction de l'exercice 3 :

```
Lemma negb_eq_true (b : bool) : negb b = true -> b = false.
Proof.
  intros H.
  destruct b as [|].
  - discriminate H.
  - reflexivity.
Qed.
```

```
Lemma add_eq_0 (n m : nat) : n + m = 0 -> n = 0 /\ m = 0.
Proof.
  intros H.
  destruct n as [| p], m as [| q].
  - split; reflexivity.
  - discriminate H.
  - discriminate H.
  - discriminate H.
Qed.
```

```
Lemma app_eq_nil {A : Type} (l1 l2 : list A) :
  l1 ++ l2 = [] -> l1 = [] /\ l2 = [].
Proof.
  intros H.
```

```

destruct l1 as [| h1 t1], l2 as [| h2 t2].
- split; reflexivity.
- discriminate H.
- discriminate H.
- discriminate H.
Qed.

```

--- * ---

Exercice 4 : discriminate et induction

On rappelle les définitions des fonctions filter, all et any, vues au TD précédent :

```

Fixpoint filter {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => []
  | h :: q => let fq := filter p q in if p h then h :: fq else fq
  end.
Fixpoint all {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => true
  | h :: q => if p h then all p q else false
  end.
Fixpoint any {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => false
  | h :: q => if p h then true else any p q
  end.

```

1. Prouver :

```

Lemma all_filter {A : Type} (p : A -> bool) (l : list A) :
  all p l = true -> filter p l = l.

```

2. Prouver :

```

Lemma any_filter {A : Type} (p : A -> bool) (l : list A) :
  any p l = true -> filter p l <> [].

```

--- * ---

Correction de l'exercice 4 :

```

Lemma all_filter {A : Type} (p : A -> bool) (l : list A) :
  all p l = true -> filter p l = l.

```

Proof.

```

induction l as [| h t IH].
- intros _; reflexivity.
- simpl. destruct (p h).
  + intros H. specialize (IH H).
    rewrite IH. reflexivity.
  + intros H. discriminate H.

```

Qed.

```

Lemma any_filter {A : Type} (p : A -> bool) (l : list A) :
  any p l = true -> filter p l <> [].

```

Proof.

```

induction l as [| h t IH].
- simpl. intros H. discriminate H.
- simpl. destruct (p h).
  + intros _ H. discriminate H.
  + exact IH.

```

Qed.

--- * ---